

TUGAS UJIAN AKHIR SEMESTER

TEKNIK KOMPILASI

Nama : Ferdy Ardiansyah

Kelas : 06TPLE003

Nim : 231011402416

Pilihan Konstruksi dipilih adalah perulangan/ Looping/while

Pattern (Pola Sintaks):

`<while_stmt> ::= "while" "(" <condition> ")" "{" <statements> "}"`

`<condition> ::= <identifier> <operator> <value>`

`<statements> ::= <identifier> "=" <identifier_or_num>`

PENJELASAN IMPLEMENTASINYA

Analisis Alur Kerja Kompilator:

Program ini mensimulasikan proses translasi kode tingkat tinggi (high-level language) menjadi kode antara (Intermediate Representation) berupa Three-Address Code (TAC).

Proses ini dibagi menjadi tiga fungsi utama di dalam kelas WhileLoopCompiler:

Source Code - lexical_analysis() - syntax_semantic_analysis() - generate_tac() - [TAC Output]

1. Inisialisasi Kelas

Pada saat objek WhileLoopCompiler dibuat, kelas akan menyimpan source_code input dan menyiapkan label_counter. Fungsi new_label() bertugas memproduksi label unik seperti L1 dan L2, secara berurutan. Label-label ini sangat krusial dalam TAC untuk menandai alamat tujuan ketika program melakukan lompatan alur.

2. Tahap Analisis Leksikal

Fungsi ini menyisipkan spasi di sekitar karakter khusus atau pembatas seperti kurung () dan kurung kurawal {} menggunakan fungsi .replace(). Setelah jaraknya renggang, fungsi .split() memotong seluruh string berdasarkan spasi tersebut.

3. Tahap Analisis Sintaksis & Semantik

Tahap ini bertugas memeriksa apakah susunan token yang dihasilkan sudah memenuhi aturan tata bahasa Grammar/BNF yang telah ditentukan, sekaligus memisahkan komponen penting di dalamnya. Cara kerjanya Fungsi melakukan validasi mendasar menggunakan kata kunci in. Struktur dianggap valid jika terdapat token.

4. Tahap Pembangkitan Kode Antara

tahap akhir adalah merombak alur perulangan tingkat tinggi yang tadinya berbentuk blok menjadi kode instruksi linear tiga alamat/TAC. Untuk membuat perulangan berjalan, kompilator membutuhkan dua label: satu untuk menandai titik awal pengecekan kondisi (L1), dan satu untuk titik keluar jika kondisi salah (L2).

Logika TAC yang Dihasilkan:

- L1: Penanda untuk kembali ke atas mengecek kondisi di setiap putaran.
- ifFalse $x < 10$ goto L2 Instruksi lompatan bersyarat. Jika kondisi $x < 10$ bernilai salah, program langsung melompat keluar ke L2.
- $x = x + 1$ Eksekusi tubuh perulangan jika kondisi masih bernilai benar.
- goto L1 Instruksi lompatan tanpa syarat untuk memaksa program memeriksa kembali kondisi di atas.
- L2: Target alamat untuk keluar dan mengakhiri perulangan while.

5. Output Evaluasi

Ketika program dijalankan dengan input while ($x < 10$) { $x = x + 1$ }, kompilator mini ini berhasil membuktikan translasinya secara akurat di Jupyter Notebook dengan output akhir berupa struktur kontrol alur linear:

L1:

ifFalse $x < 10$ goto L2

$x = x + 1$

goto L1

L2:

Struktur linear inilah yang nantinya siap diserahkan ke bagian Back-End Compiler untuk diubah secara langsung menjadi instruksi bahasa pemrograman tingkat rendah seperti Assembly.